Does Your MCP Server Actually Follow the Protocol? An Execution-Based Conformance Study of the Model Context Protocol Ecosystem

Ahmed Faraj
Faculty of Computers and Informatics, Suez Canal University
fciugs25@ci.suez.edu.eg ahmed.faraj.cs@gmail.com

Abstract

The Model Context Protocol (MCP) has become the de-facto standard for connecting LLM agents to external tools, and public registries now list tens of thousands of servers. Yet the published empirical studies of this ecosystem are *static*: they scan source code, mine issue trackers, or read registry metadata. None installs and runs the servers to ask a prior question: do these servers actually work, and do they actually follow the protocol? We answer it with an execution-based conformance study of MCP servers as installed and run. From the official registry (17,432 unique servers) we execute a complete census of all 6,106 self-contained, locally-installable servers, launching each in an isolated sandbox and driving it through a conformance harness that exercises the initialization lifecycle, tool listing, declared-schema validity, and behavior under three inputs a real agent session produces: unknown tool names, wrong-typed arguments, and malformed protocol frames. We find that only 61.1% (95% CI 59.8–62.3) of installable servers start and complete a handshake; that 88.4% (95% CI 87.3–89.4) of responding servers diverge from the specification's illustrated error-reporting behavior in an identical way, which we trace to SDK defaults and which an open specification proposal is now moving to ratify; and that 3.5% (95% CI 3.0–4.1) execute a tool on an argument their own declared schema rejects, returning a well-formed result that no client can distinguish from a correct answer, in violation of the specification's requirement that servers validate all tool inputs. We release the harness and the disclosure-filtered dataset. Conformance here is set less by server authors than by a handful of SDK defaults, so it can be measured and corrected at that layer.

1 Introduction

Large language model agents increasingly act on the world through external tools, and the Model Context Protocol (MCP) has become the dominant standard for connecting them. In under two years MCP has gone from a single vendor's proposal to an ecosystem with public registries listing tens of thousands of servers; the official registry alone contains 17,432 unique servers at the time of our study. When an agent selects a tool and an MCP server mishandles the request, by returning a malformed result, accepting ill-typed arguments, or failing to start, the agent does not merely lose a capability, and may proceed on wrong output with full confidence. The reliability of this server layer is therefore a reliability property of every agent built on top of it.

Yet we know remarkably little about how these servers actually behave, because the published empirical studies of the MCP ecosystem examine servers *without running them*. Existing work scans source code and metadata for health and maintainability, crawls registries to characterize scale and supply-chain structure, or mines GitHub issues to taxonomize faults reported by users. These static lenses answer valuable questions, but they cannot answer a prior and more basic one: *if you install these servers and speak the protocol to them, do they work, and do they follow the specification?* The protocol maintainers ship a conformance test suite, but it exercises the handful of official SDKs in continuous integration and has never been pointed at the deployed ecosystem. The official suite therefore tests the five SDKs, while the seventeen thousand servers built on them are untested.

This paper addresses that gap with an execution-based conformance study of the MCP server ecosystem. Starting from the official registry, we execute a complete census of every self-contained, locally-installable server, launch each in an isolated sandbox, and drive it through a conformance harness that exercises the initialization lifecycle, tool listing, declared-schema validity, and behavior under three inputs that arise in ordinary agent sessions: a call to an unknown tool, a call with a wrong-typed argument, and a malformed protocol frame. We grade each server against the protocol version it negotiates, and we take care to separate genuine specification requirements (RFC-2119 MUST/SHOULD) from behavior the specification merely illustrates by example.

Only 61.1% (95% CI 59.8–62.3) of installable servers start and complete a protocol handshake. The rest fail in ways we classify into a startup-failure taxonomy, from install errors to silent exits to hangs. Among servers that do respond, 88.4% (95% CI 87.3–89.4) report a call to a non-existent tool as a "successful" tool-execution error (`isError: true`) rather than the JSON-RPC protocol error the specification illustrates. We trace this near-universal divergence to the default behavior of the official SDKs, which is inherited by the servers built on them. The result bears on a live debate, in which an open proposal would reclassify unknown-tool failures to match what the SDKs already do, arguing the change aligns the specification with practice [10]. Our census says how far practice has already gone, which decides whether such a change ratifies the ecosystem or redirects it. Separately, 3.5% (95% CI 3.0–4.1) of responding servers execute a tool on an argument their own declared schema rejects and return a well-formed result. In one case a code-scanning tool asked to scan an integer replied `CLEAN`, with neither an error code nor an `isError` flag a client could act on. Unlike the error-reporting question, this behaviour violates an explicit requirement that servers validate all tool inputs. A further 0.5% (95% CI 0.3–0.7) fail to recover from a malformed frame, answering nothing further on a channel that the specification requires them to keep serving.

We organise the study around five research questions:

- **RQ1 (Runnability).** What fraction of registry-published servers can a client install and start, and when they do not start, why not?
- **RQ2 (Lifecycle conformance).** Of the servers that respond, how many correctly implement the initialization lifecycle, tool listing, and declared-schema validity?
- **RQ3 (Input enforcement).** How many servers execute a tool on an argument their own declared schema rejects, and what does the client receive when they do?
- **RQ4 (Specification versus practice).** Where the specification and deployed behaviour disagree on error reporting, how large is the divergence, what causes it, and which side is moving?
- **RQ5 (Correlates).** Do runnability and conformance vary with the package registry a server ships on?

This paper makes four contributions:

- **A protocol-level conformance measurement of the MCP ecosystem by execution** (C1): where prior ecosystem studies read code and metadata, we install and speak the protocol to a complete census of the eligible registry, testing behaviour against inputs rather than inspecting declarations, and releasing an open harness and dataset so the measurement is fully reproducible.
- **A two-dimensional taxonomy** (C2) of startup-failure classes and conformance-violation classes, each tied to a concrete agent-facing consequence.
- **A root-cause account of the spec–practice divergence** (C3): by attributing each server to its SDK, we show the dominant divergence is institutionalized in the official SDKs rather than independently reinvented.
- **Open infrastructure** (C4): a conformance harness that extends the official suite's scenarios to arbitrary deployed servers, plus the released measurement dataset, enabling the ecosystem to monitor its own conformance.

2 Background and Related Work

2.1 The Model Context Protocol

MCP standardizes how LLM agents discover and invoke external tools. A client and server negotiate a protocol version and capabilities during an `initialize` handshake. The client can then list tools (`tools/list`) and invoke them (`tools/call`). Messages follow JSON-RPC 2.0, including its error-object semantics and its reserved error codes. Most deployments run the server locally over stdio, where stdout carries protocol traffic and nothing else. The specification is versioned by date, and servers may negotiate down to a version they support.

2.2 Static studies of the MCP ecosystem

The ecosystem has been studied at scale, but never by running it. Health-and-maintainability work analyzes source and metadata for roughly 1,900 servers using static scanners and code-smell detectors [4], while registry measurement crawls several marketplaces and characterizes scale, maintenance and supply-chain concentration across some 8,000 servers, all of it derived from listing metadata rather than from anything the servers do [3]. A companion dataset effort catalogues 3,238 candidate MCP repositories on GitHub and classifies them by operational role, again from repository evidence rather than behaviour [17], and a parallel line studies the *application* side, characterizing how 1,723 MCP clients configure and supervise the servers they consume [7]. Fault taxonomies take a third route and mine GitHub issues [11, 16]. They find that tool faults and schema-enforcement problems dominate what users actually hit, which is the same pair our harness measures directly rather than through bug reports. None of this work executes a server.

2.3 Execution-based work is security-focused

Some prior work does execute MCP servers, but it targets security rather than conformance. Sandboxed behavioral scanners and taint-style analyzers hunt vulnerabilities in curated or deliberately vulnerable server sets [15, 14], a security-invariants benchmark avoids live third-party execution altogether in favor of fixtures and a 40-repository metadata sample [6], and malicious-server detection, authentication studies, and dynamic assessment of internet-facing endpoints scope to security surfaces [5, 18, 12].

Execution at scale is no longer rare. MCPZoo assembles 64,611 unique servers, of which more than 37,288 support dynamic analysis, and uses them to show that existing security scanners are unreliable, with fewer than half of sampled alerts holding up under manual validation [2]. Two differences separate that corpus from ours and they matter for what each can measure. First, MCPZoo is built by *repairing* servers: a multi-agent framework infers environments and iteratively fixes deployment and runtime defects so that in-the-wild repositories become running services. That construction is the right choice for studying what a server does once it runs, and it necessarily removes the signal we measure, which is how often a server runs at all when a client installs it as published. Second, its population is mined from GitHub repositories, whereas ours is the official registry, the surface a client actually resolves against. That their corpus required a repair framework at all is independent corroboration of our runnability finding. Conformance remains a separate question, namely whether a server that is neither malicious nor vulnerable does what the specification says.

2.4 The specification's own error-handling debate

The categorisation this paper measures is under active revision. SEP-1303, accepted and reflected in the current specification, moved tool input validation errors out of protocol errors and into tool execution errors, reasoning that a language model can self-correct only from an error that reaches its context [9]. SEP-2145, open at the time of writing, extends the same treatment to tool resolution failures, including unknown tools, and states its purpose as aligning the specification with existing Python and TypeScript SDK behaviour [10]. Both proposals argue from implementation practice. Rather than argue which mechanism is correct, we measure what fraction of the deployed ecosystem the proposed alignment already describes.

Concurrent with this work, a practitioner census posted to the same proposal probed roughly 10,500 *remote* registry endpoints, harvested their tool lists, and reported that 88.8% of mutating tools declare no output schema at all, arguing that the proposal's output-validation clause is inert for most of the write



Figure 1: Measurement pipeline. Every eligible server is installed and executed in its own disposable container, then driven through the conformance probe and graded against the protocol version it negotiates.

surface [13]. That measurement and ours are close to disjoint. It probes hosted HTTP endpoints and reads declared metadata, whereas we install and execute local stdio servers and test behaviour against inputs. It addresses the output-validation clause, and we address tool resolution and input enforcement. Both studies support the same underlying point, that this proposal has been debated without deployment data.

2.5 Conformance tooling and the gap we fill

The protocol maintainers ship a conformance suite. It exercises the official SDKs in continuous integration, and it has never been pointed at the deployed ecosystem. The community has begun proposing a pre-publish conformance checklist for server authors, which is demand for precisely this measurement. Practitioner audits circulate headline numbers, such as the claim that most listed servers are unmaintained, without published methodology, data, or protocol-level testing. Ours is, to our knowledge, the first reproducible conformance measurement of this ecosystem that installs and executes the servers themselves and tests their behaviour against adversarial inputs, released with an open harness and dataset. It follows a long line of empirical software-engineering studies that execute a population to find out how it behaves, rather than how its documentation says it should.

3 Methodology

We measure the MCP server ecosystem by *executing* it. This section defines the sampling frame, the conformance harness, the sandbox, and the grading rules (Figure 1). The harness and the released dataset make every number reproducible.

3.1 Sampling frame

We crawl the official MCP registry (`registry.modelcontextprotocol.io`) via its cursor-paginated API, retaining the latest published version of each server. This yields 17,432 unique servers. Each server declares zero or more *packages* (installable artifacts on npm, PyPI, OCI, NuGet, or Cargo) and/or *remotes* (hosted HTTP endpoints). Our study population is the set of servers that are (i) locally installable, (ii) use the `stdio` transport, (iii) distributed on npm or PyPI (the two dominant registries), and (iv) *self-contained*: they declare no required environment variable or argument that lacks a default. Criterion (iv) restricts us to servers that a client could launch without provisioning secrets, which both bounds the ethics of mass execution and defines a clean, reproducible population. We report the exclusion of remote-only and credential-requiring servers explicitly (Section 6) and treat the resulting frame as the denominator for all rates. We probe the *entire* frame (all 6,106 eligible servers), so our measurement is a complete census of the eligible population rather than a sample.

3.2 Conformance harness

The harness, `mcpprobe`, is a deliberately minimal, hand-written JSON-RPC client speaking line-delimited messages over the server's stdio. We do *not* use an official SDK client. To observe raw behavior, including responses to malformed frames, the probe must not inherit any SDK-side correction or validation. The harness offers protocol version `2025-06-18` at initialization, records the version the server *negotiates*, and grades all subsequent checks against that negotiated version rather than the latest specification. This is not a formality. Responding servers negotiated four distinct revisions, and 109 of them settled on something other than the version we offered, the oldest dating to 2024-11-05.

Each server is driven through the following checks, whose identifiers align with the categories of the official `modelcontextprotocol/conformance` suite where an analogue exists:

- **server-initialize**: the `initialize` request must return a result carrying a protocol version and a capabilities object; the harness then sends `notifications/initialized`.
- **ping**: a `ping` must return an empty-object result.
- **tools-list**: if the server advertises the tools capability, `tools/list` must return an array in which every entry has a `name` and an `inputSchema` object.
- **tools-schema-valid**: each declared `inputSchema` must be a valid JSON Schema (checked by meta-validation).
- **tools-call-unknown**: calling a non-existent tool must be rejected. The specification's Error Handling section categorizes "Unknown tools" under *Protocol Errors* (standard JSON-RPC errors) and illustrates this with a `-32602` example, as opposed to *Tool Execution Errors* reported as a result with `isError: true`. This categorization is prose plus an example; it uses no RFC-2119 MUST/SHOULD keyword. We therefore do not treat a non-conforming response as a "violation"; instead we record which of the two mechanisms the server uses (see error-as-result below).
- **tools-call-invalid-args**: for a tool with a typed required property, calling it with a wrong-typed value should be rejected. The spec is itself ambiguous here, since it lists "Invalid arguments" under Protocol Errors but "Invalid input data" under Tool Execution Errors, so we count *either* a protocol error *or* an `isError` result as acceptable, and flag only a plain success response (the tool ran on ill-typed input) as a defect.
- **malformed-json**: after receiving a syntactically invalid frame, the server must not crash or hang; a subsequent `ping` must still succeed.
- **stdout-purity**: under the stdio transport, the server must emit only protocol messages on stdout; any non-JSON line is a channel-corruption defect.

3.3 Grading and the error-as-result category

Verdicts are pass, fail, warn, skip, and one category introduced by calibration: error-as-result. When validating the harness against the official reference servers (`server-memory`, `server-everything`), we found they answer an unknown-tool call not with the specified JSON-RPC `-32602` error but with a *successful* response whose `result.isError` flag is set. Because this behavior originates in the official SDKs and is therefore ecosystem-wide, we grade it as its own category rather than a failure, and quantify its prevalence as a first-class result, since it is the clearest instance of specification text diverging from universal practice.

3.4 Sandbox

Executing thousands of untrusted packages demands isolation. Each server runs in a fresh container (`node:22-slim` for npm, an `astral-sh/uv` image for PyPI) with a memory cap, a single CPU, a PID limit, `no-new-privileges`, all Linux capabilities dropped, and no host filesystem mounts other than a package-cache volume. The harness supports two probe conditions. In the **two-phase** condition, an online

install pass populates the cache and the conformance probe then runs with `--network=none`, so no server code has network access while it is being measured. In the **single-phase** condition, install and probe occur together in one ephemeral container and the network is available throughout. The census reported here was measured in the single-phase condition, for the operational reasons given in Section 3.5, and we state the consequences for validity in Section 6.

3.5 Execution environment and probe condition

The census was executed on a single disposable cloud instance (no cloud credentials attached, so the blast radius of running untrusted code is one throwaway host) using the online single-phase condition: install and probe occur in one ephemeral `--rm` container per server, with nothing cached between servers so that disk usage stays bounded across the full ecosystem. In a pilot on an independent subset (roughly 200 servers per arm) we compared this condition against the two-phase network-isolated condition and found no detectable difference on any metric: handshake yield (54% vs. 57%), error-as-result prevalence (87% vs. 87%), and silent type non-enforcement (7% vs. 7%). We note that a pilot of this size can only exclude differences larger than roughly 14 percentage points in handshake yield, so this is evidence of consistency rather than a demonstration of equivalence; the conformance metrics, being far more tightly clustered, are correspondingly better constrained. A small number of probes ($\leq 0.4\%$) were interrupted by an automated disk-guard that reclaims space if the host approaches capacity; these are recorded conservatively as non-handshaking and, being a property of the harness environment rather than the server, do not bias conformance rates (which are computed over responding servers).

3.6 SDK-level confirmation

Where a conformance failure concentrates in one SDK family, the census alone cannot say whether the library permits the failure or the authors introduced it independently. For the input-validation result we settle this with a direct test rather than inference. We build two minimal servers exposing one tool that declares a single required string property, one using the low-level `Server` API and one using the high-level `McpServer` API, both on the same SDK release, and drive each with the same harness check used in the census: a `tools/call` carrying an integer where the declared schema requires a string. The construction is otherwise identical, so any difference in outcome is a property of the API path. We report the result in Section 4.3. The two servers and the probe script are released with the harness.

3.7 Funnel accounting

Because "does it even run" is itself a research question, we count every stage: declared → eligible (self-contained stdio npm/PyPI) → sampled → container started → process launched → handshake completed → probed. Servers that never reach a handshake are classified into startup-failure categories (Section 4) from their exit code and stderr, so that the yield loss is characterized rather than merely reported.

4 Results

We probed 6,106 servers, the complete eligible frame, which makes this study a census rather than a sample.

4.1 RQ1: Runnability and the startup funnel

3,729 of 6,106 servers (61.1% (95% CI 59.8–62.3)) complete the MCP handshake once the entry-point artifact described in Section 4.5 is corrected; the uncorrected census figure is 60.4% (95% CI 59.1–61.6). Roughly two in five never serve the protocol at all. Figure 2 classifies those failures by cause.

Two distinctions matter for reading that figure. First, servers that merely need credentials or configuration are counted separately from servers that crash; conflating them would blame the ecosystem for what is a documentation gap. Second, most failures occur *after* a successful install, when the package downloads and the process then crashes, hangs, or exits without speaking the protocol, which locates the failure in the

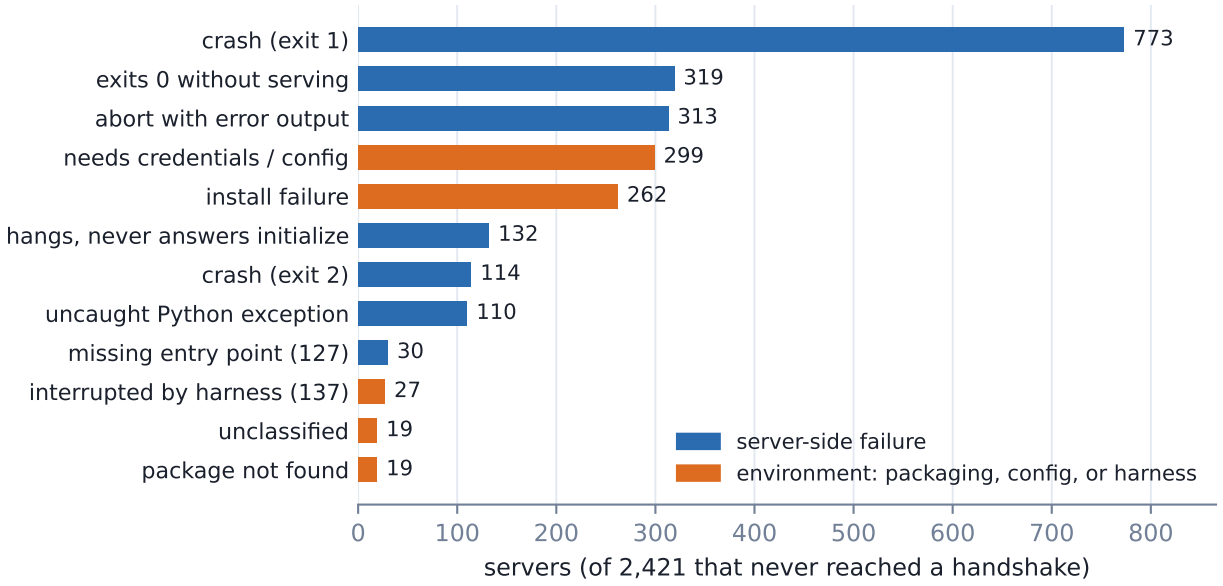


Figure 2: Why servers never reach a handshake. Configuration requirements and the entry-point artifact are separated from genuine startup failures.

software rather than the packaging. No prior work has measured how often registry-published servers fail to start, and because we census the entire eligible frame, these are population counts rather than estimates.

4.2 RQ2–RQ3: Conformance and robustness

Table 2 reports the verdict distribution for each check over the 3,685 responding servers, with 95% Wilson confidence intervals. Lifecycle and tool-listing conformance are high, and the discriminating checks are error handling, argument type enforcement, and malformed-input survival. 7.4% (95% CI 6.6–8.3) of responders failed to reject a wrong-typed required argument, which Section 4.3 decomposes into the servers that reported the problem anyway and the smaller class that did not. 0.5% (95% CI 0.3–0.7) failed to recover from a malformed frame. We report this as a robustness and conformance failure rather than a security one. Under the stdio transport the client is the parent process and the only writer to the server's standard input, so a party able to deliver a malformed frame can already terminate the subprocess directly; the specification nonetheless requires the server to answer a parse error and continue serving, and these servers do not. One maintainer who reproduced our finding made exactly this argument, and we adopt their characterization over our original one (Section 6).

4.3 RQ3 refined: what a failure to reject actually returns

The `tools-call-invalid-args` verdict records whether a server rejects an argument that violates its own declared schema. That is the right protocol judgement, but it is too coarse for the question a client faces, because two different behaviours produce the same verdict. A server may validate the input and report the problem in the result *text* without setting `isError`, which is non-conformant but leaves the information available. Or it may execute the tool and return ordinary-looking output, which does not. We separate the two by classifying the recorded response of every server in this category (Figure 3, Table 3).

Of 273 such servers, 118 (43%) reported the problem in the result text, so the server noticed and a client willing to parse prose could too. 129 (47%) executed the tool and returned a result with no indication that anything was wrong. For 26 the response could not be classified from the transcript. The hazard class is therefore 3.5% (95% CI 3.0–4.1) of responding servers rather than the 7.4% (95% CI 6.6–8.3) implied by the raw verdict, and we report the smaller figure as the one that matters. We count a server as reporting the

Table 1: Startup-failure taxonomy: causes for servers that never reach a handshake.

Failure class	Count	% of failures
crash-exit-1	773	31.9
exit-silent	319	13.2
crash-with-error-output	313	12.9
needs-auth-or-config	299	12.4
install-error	262	10.8
hang-no-reply	132	5.5
crash-exit-2	114	4.7
crash-python-exception	110	4.5
crash-exit-127	30	1.2
crash-exit-137	27	1.1
unknown	19	0.8
install-not-found	19	0.8
crash-exit-255	1	0.0
crash-exit-3	1	0.0
crash-exit-4	1	0.0
crash-exit-254	1	0.0
Total non-handshaking	2421	100.0

Table 2: Verdict distribution per check over responding servers, with 95% Wilson CIs on the pass (or documented) rate.

Check	Pass	Fail	Other	Pass rate (95% CI)
server-initialize	3685	0	0	100.0 (99.9–100.0)
ping	3638	47	0	98.7 (98.3–99.0)
tools-list	3673	5	7	99.7 (99.4–99.8)
tools-schema-valid	3671	2	0	99.6 (99.4–99.8)
tools-call-unknown	139	166	3380	3.8 (3.2–4.4)
tools-call-invalid-args	3309	273	103	89.8 (88.8–90.7)
malformed-json	3668	17	0	99.5 (99.3–99.7)
stdout-purity	3654	31	0	99.2 (98.8–99.4)

problem whenever its output contains any error-like token, so 129 is a lower bound.

Unlike the error-reporting question, this one is not a matter of interpretation. The specification's security requirements state that servers **MUST** validate all tool inputs [8]. A server that executes a tool on an argument contradicting its own published schema has not validated that input, so these 129 servers are the one population in our census that fails an explicit RFC-2119 requirement rather than diverging from an illustrated convention.

Attributing these servers to their SDK localises the cause more sharply than the error-reporting divergence does. 112 of the 129 are built on the official TypeScript SDK and 0 on either Python SDK, against responding populations of 2,347 and 1,020 respectively. A behaviour absent from one language ecosystem and concentrated in another is a library property rather than an authoring one, so we tested the SDKs directly (Section 3.6). On `@modelcontextprotocol/sdk` 1.30.0, the current release at the time of writing, a server built with the low-level `Server` API publishes an `inputSchema` in `tools/list` and the SDK then does not check `tools/call` arguments against it: a tool declaring a string property, called with an integer, runs and returns a well-formed result. The same tool built with the high-level `McpServer` API, whose schemas carry type information the SDK already validates, rejects the identical call with an `isError` result. We did not test the Python SDKs under the same conditions, so we report their absence from this population as an observation rather than attributing a mechanism to it. Within the TypeScript SDK the gap is that one supported construction path publishes a contract the library never enforces, so the servers on it have not

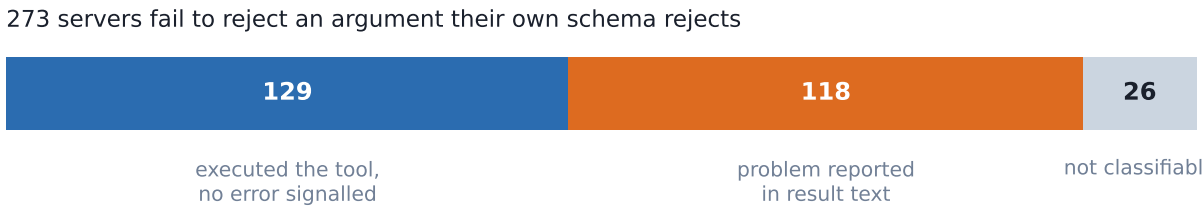


Figure 3: The coarse verdict splits in half. Only the left segment is undetectable by the client: the tool ran on input its schema rejects and returned a well-formed result carrying no error indication.

Table 3: Servers that executed a tool on an argument their own declared schema rejects, and returned ordinary-looking output. Inputs are the harness's type-poisoned values; outputs are verbatim, truncated. None carries an error indication a client could act on.

Tool	Argument sent	Result returned to the client
`scan_code_imports`	`code: 12345`	"CLEAN — No npm package imports found in the provided code"
`register_project`	`name: 12345`	"Project 12345 registered successfully. Framework: generic"
`score_prompt`	`prompt: 12345`	a complete scoring rubric: `score: 8`, `grade: "F"`, with per-dimension sub-scores
`upcoming_deadlines`	`within_days: "not-a-number"`	a dated regulatory snapshot with a populated result envelope
`search`	`query: 12345`	`{"results":[]}`

declined a check the SDK offered them. Of the 112 TypeScript-SDK servers, 109 declare a caret range and none declares an exact version. Because each server was installed fresh, those ranges resolve to the newest 1.x, so this is current behaviour rather than a legacy artifact, and a fix on the 1.x line would reach almost all of them at their next install.

The recorded responses show what this means in practice. A code-scanning tool asked to scan the integer `12345` answered `CLEAN`, a prompt-scoring tool returned a complete rubric including per-dimension sub-scores for the same integer, and a project registry confirmed that a project named `12345` had been registered. These are well-formed successful results rather than malformed responses a client could detect and discard. Their content is unrelated to any question the client asked, and at the protocol level they are indistinguishable from a correct answer. An agent consuming one has no signal to doubt it, since there is neither an error code nor an `isError` flag to act on, and the schema that would have caught the call is the one the server published and did not enforce.

4.4 RQ4: Where practice and specification disagree, and which one moved

88.4% (95% CI 87.3–89.4) of responding servers reported a call to a non-existent tool as an `isError` tool-execution result rather than a JSON-RPC protocol error. The specification's error-handling section lists "unknown tool" under *Protocol Errors* and illustrates it with a `-32602` example [8]. It attaches no RFC-2119 keyword, so we record which mechanism each server uses rather than scoring one as a violation (Section 6).

The relevant question here is which side is moving. The specification has been converging on the ecosystem, not the reverse. SEP-1303, now accepted and reflected in the specification, reclassified input validation errors from protocol errors to tool execution errors on the grounds that a model can only self-correct from an error it can see [9]. SEP-2145, open at the time of writing, proposes the same move for tool resolution failures, covering unknown and disabled tools, and gives as its rationale that it "aligns specification with existing Python and TypeScript SDK behavior" [10]. Our census supplies the quantity that argument has

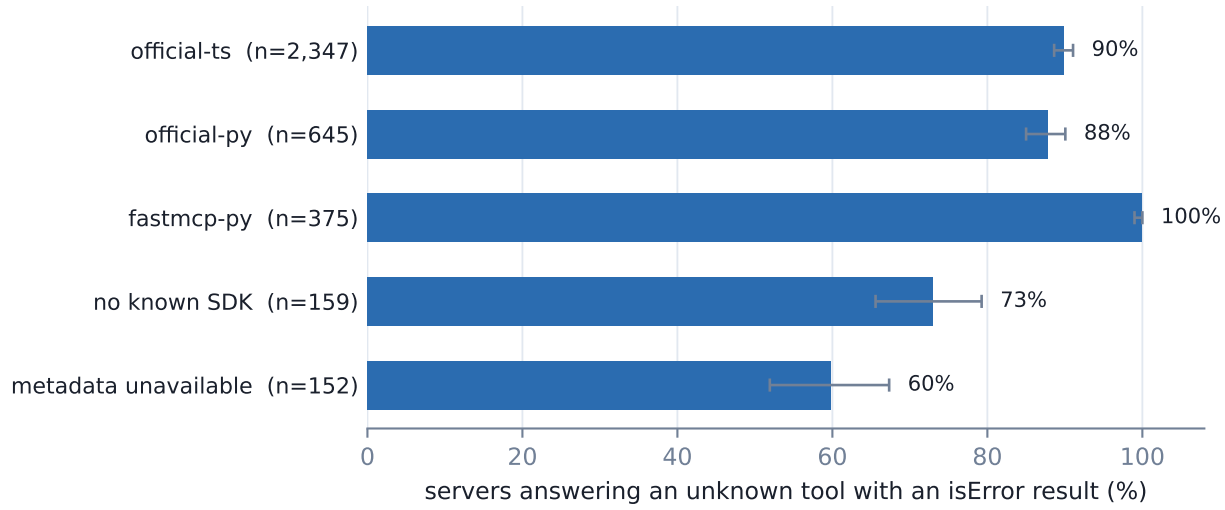


Figure 4: Which mechanism a server uses tracks its SDK, not its author. Every SDK family answers an unknown-tool call with an `isError` result rather than a protocol error in the large majority of cases. Bars show 95% Wilson intervals; families with fewer than 25 servers appear in Table 4 only.

Table 4: SDK family vs. unknown-tool response. Which mechanism a server uses tracks the SDK it was built on, not the author.

SDK family	error-as-result	protocol error	other
official-ts	2110	68	169
official-py	566	0	79
fastmcp-py	375	0	0
no-known-SDK	116	28	15
metadata-unavailable	91	36	25
fastmcp-ts	0	6	0
xmcp-ts	0	1	0

so far lacked, namely that the alignment it describes already covers 88.4% (95% CI 87.3–89.4) of deployed servers. If SEP-2145 is adopted it ratifies the ecosystem's existing behaviour rather than changing it, and the residual 11.6%, which are the servers emitting protocol errors, become the population that would need to migrate.

To explain the uniformity, we attribute each server to its SDK from package metadata (Figure 4, Table 4). The behaviour tracks the SDK: the official TypeScript and Python SDKs and the FastMCP framework all default to it. A small number of TypeScript-SDK servers emit the protocol error, so the alternative is reachable within the SDK and simply is not the default. The official reference servers behave the same way.

Our attribution recognizes a fixed list of known SDK package names, so the residual no-known-SDK category must not be read as "hand-rolled." Inspecting a random sample of that category, we find roughly two thirds in fact depend on an MCP framework outside our list. These are several third-party wrappers, each used across dozens of servers by a single publisher, and most of them themselves build on an official SDK. That category therefore supplies no independent evidence and we do not treat it as such. What it does indicate is that the monoculture extends *beyond* the official SDKs, into a layer of third-party wrappers that inherit their defaults. The finding is that a handful of SDK defaults, propagated both directly and through wrappers, set ecosystem behavior, rather than thousands of authors independently erring.

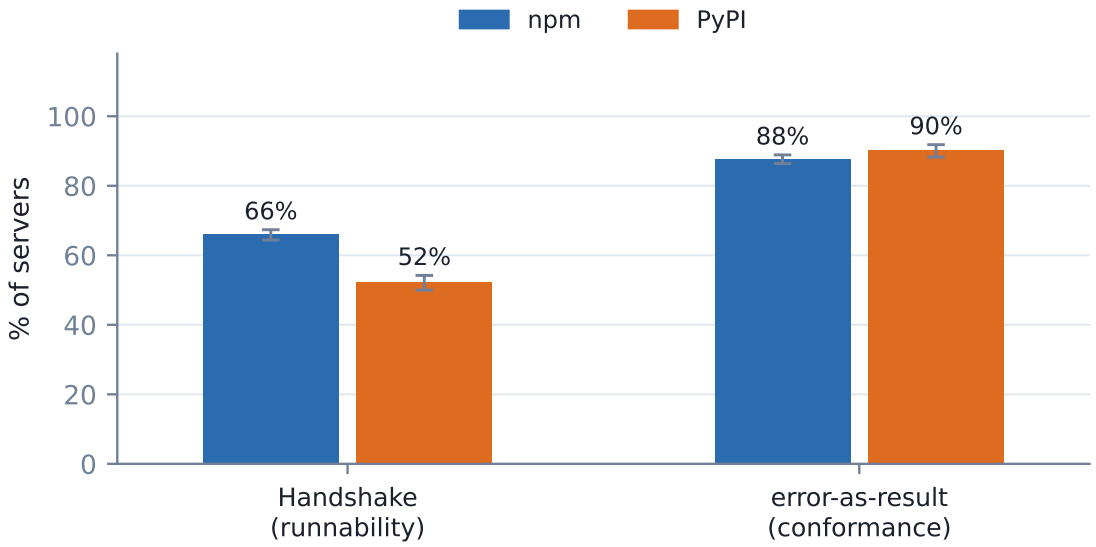


Figure 5: Handshake yield and conformance-check pass rates by package registry, with 95% confidence intervals.

4.5 RQ5: Correlates

We test whether runnability and conformance correlate with the package registry (Figure 5). Runnability differs sharply. npm servers complete a handshake far more often than PyPI servers (65.9% of 3,968 vs. 50.0% of 2,138; two-proportion $z = 12.1$, $p < 10^{-3}$). Conformance does not differ, with the error-as-result divergence appearing in 87.7% of responding npm servers and 90.2% of responding PyPI servers. The pattern follows from RQ4: because the divergence is inherited from the SDKs, and both language ecosystems' SDKs share the same default, the registry a server ships on changes whether it *starts* but not whether it *conforms*.

Part of this gap is an artifact of our own launcher, and we corrected it rather than reporting it. We launch PyPI servers as `uvx <package>`, which resolves only if the package's console script is named after the package; npm's `npx` resolves the entry point from `package.json` and has no such requirement. 462 PyPI servers (21.6% of the PyPI population) installed successfully but were never launched for this reason alone. We re-probed all of them with the entry point resolved from the installed package metadata. 44 (9.5% (95% CI 7.2–12.5)) then completed a handshake; the rest failed for reasons unrelated to naming (Python exceptions, hangs, undeclared configuration). Correcting for this raises PyPI runnability from 50.0% to 52.1% (95% CI 50.0–54.2) and the overall census figure to 61.1% (95% CI 59.8–62.3), which we report as the headline throughout. The npm–PyPI gap narrows from 15.9 to 13.8 percentage points but remains large, so it reflects a real difference rather than a measurement artifact.

The entry-point problem is itself an ecosystem finding. Registry metadata does not record a server's entry point, so a client following the registry alone cannot launch a fifth of published PyPI servers, and once the entry point is resolved, nine in ten of those servers still fail to start. Discoverability and runnability are therefore distinct failures, and the registry surfaces neither.

5 Discussion

The specification is following the ecosystem, not leading it. On the question our largest measurement covers, the standard is being rewritten to match what implementations already do. SEP-1303 moved input validation errors to tool execution errors, and SEP-2145 proposes the same for unknown tools, explicitly to align the specification with SDK behaviour [9, 10]. The reasoning is sound, since an error the model can read is one it can recover from, whereas a protocol error never reaches it. What the debate has lacked is a measurement of how far the ecosystem has already gone. It has gone almost all the way, to 88.4%

(95% CI 87.3–89.4) of deployed servers, which means SEP-2145 would ratify existing behaviour rather than impose new behaviour, and the migration cost falls on the conforming minority instead. That is a different decision from the one the proposal appears to be making, and deciding it requires deployment data rather than further specification discussion.

The SDK is the unit of change. Attributing servers to their SDK collapses thousands of independent authorship decisions into a handful of library defaults, and the residual category shows those defaults propagating a second time through third-party wrappers. The effect runs in both directions, since a poor default reaches everyone and so does a fix. Changing a few SDKs would move conformance for the whole population at once, which is why the SEP process treats SDK behaviour as a first-class input rather than an implementation detail. It also means ecosystem conformance should be measured at the SDK layer, where a single change is worth thousands of server-level ones.

Input validation is not a matter of interpretation. The unknown-tool question is genuinely contested, and we decline to score it, but the input-validation result stands on different ground. The specification requires servers to validate all tool inputs, and 129 servers run tools on arguments their own published schema rejects, returning results that carry no sign of it. No proposal is pending to relax that requirement, and no client-side convention recovers the information once it is gone. Where a specification uses MUST and practice ignores it, only measurement surfaces the gap, because nothing in the registry, the package metadata, or a static scan reveals which servers enforce their own schemas.

Startup failure is invisible to the registry. Two in five installable servers never serve the protocol, and a fifth of PyPI servers cannot be launched from registry metadata alone because that metadata does not record an entry point. Registries rank servers by popularity and recency, and none of them surfaces whether a server starts. Server selection is an empirical reliability question that the available metadata cannot answer, and a pre-publish conformance check, which our harness implements, would turn a one-time measurement into a standing signal.

Recommendations. For the specification: SEP-2145 should be decided with the deployment numbers in hand, because they change what the proposal is doing. For SDK authors: input validation is where the normative requirement actually bites, and a default that validates declared argument types before dispatch would close the one gap in this study that is unambiguously a violation. For agent frameworks: validate arguments against the declared schema client-side before dispatch, since 3.5% (95% CI 3.0–4.1) of responding servers will not, and their failure to do so produces no visible error.

6 Threats to Validity

Construct validity. Our verdicts operationalize "conformance," and a poorly chosen operationalization could over- or under-count. We mitigate this in three ways. First, we grade each server against the protocol version it *negotiates*, not the latest specification, so a server is never penalized for a requirement that postdates its target revision. Second, we distinguish normative requirements from illustrative ones, and the most prevalent divergence we report (unknown-tool responses) is graded as a distinct error-as-result category rather than a failure, precisely because the specification expresses it in prose and an example rather than an RFC-2119 keyword. Third, we calibrated the harness against the official reference servers before the study and confirmed it reproduces their known-good behavior on every check except the SDK-level divergence we set out to measure.

Internal validity. The census was measured in the single-phase condition, with the network available during both install and probe (Section 3.5). This avoids penalizing a server that legitimately contacts the network during initialization, but it does not remove server-side outbound calls as a confounder. The exposure is concentrated in one check. For tools-call-invalid-args we accept an `isError` result as evidence that the server rejected the ill-typed argument; an `isError` produced instead by an unrelated runtime or network failure is indistinguishable to the harness, and would be counted as argument checking that did not

in fact occur. The reported rate of silent type non-enforcement is therefore a *lower bound*. The lifecycle, tool-listing, error-handling, malformed-input and stdout-purity checks are local protocol behaviors and are not exposed to this confound. A network-isolated replication would tighten the argument-checking result, and the harness supports that condition directly. Sandbox heterogeneity across language runtimes is a further source of noise, which we bound by pinning base images and package versions and by stamping every result with the harness commit that produced it.

External validity. Our population is self-contained, locally-installable servers distributed on npm and PyPI using the stdio transport. This excludes remote-only servers (which we cannot execute without provisioning hosted infrastructure) and servers requiring credentials (which we exclude for both ethics and reproducibility). We report the size of each excluded class explicitly and confine all rates to the stated frame; we do not claim they generalize to remote or credentialed servers, whose conformance is important future work. Because agent clients overwhelmingly launch local stdio servers, we argue this population is the most consequential one to characterize first.

Completeness. We probe the entire eligible frame, all 6,106 servers (3,685 responding), so the study is a complete census of the eligible population rather than a sample. The confidence intervals we report throughout therefore reflect residual measurement noise (e.g. server-side nondeterminism) rather than sampling error, and should be read together with the publisher-clustering caveat below. The conformance findings (RQ2–RQ4) are near-universal, and the runnability funnel (RQ1) rests on the full population rather than an estimate.

Publisher clustering. The 6,106 servers in our frame come from only 3,913 distinct registry namespaces, and the distribution is heavy-tailed: the largest single publisher accounts for 305 servers (5.0% of the census) and the ten largest for 12.6%. Servers from one publisher frequently share a template and behave near-identically, so they are not independent observations, and the Wilson intervals we report, which assume independent trials, understate the true uncertainty. To bound the effect, we recompute each headline rate on a publisher-deduplicated subset retaining one server per namespace (3,913 servers). Every estimate moves by a few points and no conclusion changes: handshake yield 58.5% (95% CI 56.9–60.0) (against 60.4% (95% CI 59.1–61.6) for the full census), error-as-result 85.2% (95% CI 83.7–86.6) (against 88.4% (95% CI 87.3–89.4)), and silent type non-enforcement 8.1% (95% CI 7.0–9.3) (against 7.4% (95% CI 6.6–8.3)). The single largest publisher is atypical in runnability (a 26.2% handshake yield against 60.4% overall), and alone depresses the census figure by 1.8 percentage points. We report the full-census figures as our primary result, because the census rather than a sample of publishers is the object of study, and the publisher-deduplicated figures as the robustness check.

Severity of the malformed-frame finding, and a limitation it exposed. We initially characterized failure to recover from a malformed frame as a denial-of-service primitive. A maintainer who reproduced the finding independently argued that this overstates it, and we agree. Under the stdio transport the client launches the server and is the sole writer to its standard input, so a party capable of sending a malformed frame can already terminate the subprocess; the specification also places the obligation not to send invalid messages on the client. The correct reading is a robustness and conformance failure, which is how we now report it, and we record the correction here rather than silently revising.

The same maintainer's reproduction exposed a limitation in our instrument. They observed the server process staying alive while its parser absorbed subsequent frames, so the following `ping` timed out. Our harness records a single verdict for "did not answer after a malformed frame" and does not distinguish a terminated process from a desynchronized one. Both are conformance failures, but they are different failures with different fixes, and 0.5% (95% CI 0.3–0.7) therefore aggregates the two. Separating them requires per-server process-state tracking that the released transcripts support but the current verdict does not encode.

Interrupted probes. A small fraction ($\leq 0.4\%$) of probes were interrupted by the automated disk-guard that reclaims host space near capacity (Section 3.5). These are recorded conservatively as non-handshaking; because the interruption is a property of the harness environment and not of the server, and because it is

uncorrelated with any server property, it does not bias conformance rates (computed over responding servers) and can only very slightly depress the handshake yield. The released dataset flags these runs for audit.

Reproducibility. Every number in this paper is regenerable from released raw JSON-RPC transcripts and the pinned harness commit. The sampling seed, registry snapshot, base images, and package versions are all recorded. Because the registry is a moving target, we report the crawl date and treat the snapshot, not the live registry, as the object of study. A longitudinal measurement of the same registry quantifies how fast that target moves, reconstructing 120 observations over 88.6 days as the registry grew from 3,510 to 18,966 servers, and finds that the descriptive surface drifts unevenly, with 8.6% of servers ever rewriting a description [1]. That work concerns registry text rather than server behaviour, so it does not date our conformance results directly, but it is the reason we fix a snapshot and report its date rather than describing the registry in the present tense.

7 Ethics and Responsible Disclosure

This study executes third-party code and probes servers for defects, so we adopt the norms of responsible measurement research.

Safe execution. Every server, during both installation and probing, runs inside an isolated container with all Linux capabilities dropped, no new privileges, bounded memory, CPU, and process counts, and no host filesystem access beyond a package cache volume. The census was executed in the single-phase condition, in which install and probe share one ephemeral container and the network is available throughout; the install phase, which executes arbitrary `postinstall` and build scripts, therefore receives the same isolation as the probe. We are explicit that server code had network access while being measured, and we mitigate the resulting risk by structural means rather than by network policy. Every container is discarded after a single server, and the entire census ran on a disposable cloud instance with no credentials attached, so the blast radius of executing untrusted code is one throwaway host. The harness also supports a network-isolated probe phase (`--network=none`), which we recommend for replications that do not need install and probe in a single pass. We execute no server that declares a requirement for credentials, both because it lies outside our frame and because doing so could send real requests to third-party systems.

Observed side effects of networked measurement. Because the census ran with the network available, we audited the captured output for evidence that server code produced effects outside its container. We report what we found rather than only what we intended. Roughly 1% of servers attempted an outbound connection during startup, most of which failed. Five servers emitted live credential material into their own logs (session tokens they had generated), and one auto-provisioned an API key and account against a third-party service, which is precisely the class of effect our frame's exclusion of credential-requiring servers was meant to prevent. We have redacted every such value from the released dataset, notified the affected service, and requested revocation of the provisioned key. We draw two conclusions. First, excluding servers that *declare* required credentials does not exclude servers that *acquire* them unprompted, so declaration-based eligibility filtering is a weaker ethical guarantee than it appears. Second, a network-isolated probe phase should be the default for measurement at this scale, and we recommend replications use the harness's two-phase mode. Our own use of the single-phase condition was an operational concession to running an entire ecosystem on one disposable host, and we state its cost explicitly here.

Responsible disclosure. A subset of our findings concern robustness rather than function. A server that stops serving after a malformed frame, and a server that emits non-protocol bytes on its stdio channel, can each desynchronize or strand a connected client. We report these only in aggregate and through anonymized case studies. We do not publish a name-and-shame list of individual servers. Security-relevant findings are recorded in a private disclosure log with maintainer contacts resolved from registry repository metadata, and were reported to maintainers before publication. Of the servers whose maintainers we could reach, two reproduced the behaviour independently within a day: one shipped a fix in a released version, and one confirmed a fix for its next release with a regression test. A third response corrected our severity assessment,

which we adopted (Section 6). We take these as evidence that the findings are actionable and that the reporting format was usable; where a finding is exploitable, per-server details are withheld until a reasonable disclosure window has elapsed. The released dataset is filtered accordingly.

Registry and maintainer impact. Our crawl uses the registry's public API within its rate limits, and our installs draw from public package registries in the ordinary way. We cache aggressively to avoid redundant load. We view the released harness as a benefit to maintainers, since it lets any author check their own server's conformance before publishing, which is the outcome the community's proposed pre-publish conformance checklist seeks.

8 Conclusion

The MCP server ecosystem has been measured statically many times, but never by running it. In this study every eligible server in the official registry was installed, launched in a disposable sandbox, and driven through the protocol.

61.1% (95% CI 59.8–62.3) complete a handshake. The rest crash, hang, or exit without serving anything at all. Among the servers that do respond, 88.4% (95% CI 87.3–89.4) report an unknown tool the same way, and that uniformity reflects the SDKs rather than the developers, since it is inherited from a handful of SDK defaults propagated directly and through third-party wrappers. The specification is currently being amended to match them, and our number tells the people making that decision whether they are ratifying the ecosystem or asking most of it to change. A smaller group, 3.5% (95% CI 3.0–4.1), will run a tool on input their published schema rejects and return a well-formed answer carrying no sign the input was never valid. One asked to scan an integer for unsafe imports reported it clean. For this group the specification is unambiguous, since it requires servers to validate their inputs, and these servers do not.

None of this is visible to a static scanner or a registry crawl, and all of it matters to anyone building on this layer. We release the harness and the dataset so the measurement can be repeated, extended to remote and credentialed servers, and run by authors against their own servers before they publish. The behaviour is set at the SDK layer rather than by thousands of independent choices, which means it can also be corrected there.

Data and code availability. The conformance harness, the analysis pipeline, and the disclosure-filtered census dataset are available at `https://github.com/Ahmad-Faraj/mcp-conformance`, with the harness pinned at tag `harness-v1.0`. The release contains one record per probed server (every verdict behind every number in this paper), together with the registry snapshot that defines the sampling frame, so the eligibility funnel and all reported rates can be recomputed independently. Two filters are applied and documented with the data: the identities of 211 servers carrying security-relevant findings are withheld pending disclosure, and credential material that servers emitted at runtime is redacted. Neither filter alters any verdict or count.

AI assistance. This work made substantial use of an AI assistant, including implementation of the measurement harness and analysis code and drafting of this manuscript. The author directed the work, reviewed and approved the methodology and findings, and takes full responsibility for its content. All code, raw transcripts, and analysis scripts are released so that every number reported here can be independently regenerated.

References

[1] Gautam Bharti. Registry descriptions go stale unevenly: An 89-day measurement of model context protocol drift, and why drift-ranked re-auditing under-covers it. *arXiv preprint arXiv:2608.00997*, 2026.

[2] Pei Chen, Baichao An, Mengying Wu, Binwang Wan, Geng Hong, Jinsong Chen, Xudong Pan, Jiarun Dai, and Min Yang. Rethinking MCP security: A large-scale study of runtime MCP servers and security scanner reliability. *arXiv preprint arXiv:2607.11086*, 2026.

[3] Hechuan Guo, Yongle Hao, Yue Zhang, Minghui Xu, Peizhuo Lv, Jiezhi Chen, and Xiuzhen Cheng. A measurement study of model context protocol ecosystem. *arXiv preprint arXiv:2509.25292*, 2025.

[4] Mohammed Mehedi Hasan, Hao Li, Emad Fallahzadeh, Gopi Krishnan Rajbahadur, Bram Adams, and Ahmed E. Hassan. Model context protocol (mcp) at first glance: Studying the security and maintainability of mcp servers. *arXiv preprint arXiv:2506.13538*, 2025.

[5] Yiheng Huang, Zhijia Zhao, Bihuan Chen, Susheng Wu, Zhuotong Zhou, Yiheng Cao, Xin Hu, and Xin Peng. From component manipulation to system compromise: Understanding and detecting malicious mcp servers. *arXiv preprint arXiv:2604.01905*, 2026.

[6] Ting Liu. From tool connection to execution control: Benchmarking security invariants in mcp-style agent runtimes. *arXiv preprint arXiv:2606.29073*, 2026.

[7] Muhammad Hamza Arshad Majeed, May Mahmoud, and Sarah Nadi. An empirical study of model context protocol applications. *arXiv preprint arXiv:2607.25635*, 2026.

[8] Model Context Protocol. Model context protocol specification: Tools, 2026. https://modelcontextprotocol.io/specification/draft/server/tools.

[9] Model Context Protocol contributors. Sep-1303: Input validation errors as tool execution errors, 2025. Accepted. https://github.com/modelcontextprotocol/modelcontextprotocol/issues/1303.

[10] Model Context Protocol contributors. Sep-2145: Standardize `tools/call` failure reporting, 2026. Open proposal. https://github.com/modelcontextprotocol/modelcontextprotocol/pull/2145.

[11] Joshua Owotogbe, Indika Kumara, Willem-Jan van den Heuvel, Damian Andrew Tamburri, Antonio Ken Iannillo, and Roberto Natella. A taxonomy of runtime faults in model context protocol servers. *arXiv preprint arXiv:2606.05339*, 2026.

[12] Nicolás Padilla. Exposed by design: A dynamic security assessment of internet-facing MCP servers at scale. *arXiv preprint arXiv:2608.00150*, 2026.

[13] siliroid. Registry census of declared output schemas on write tools, 2026. Comment on SEP-2145, 29 July 2026. https://github.com/modelcontextprotocol/modelcontextprotocol/pull/2145.

[14] Pengyu Sun, Qishu Jin, Enhao Huang, Zifeng Kang, Xin Liu, Dakun Shen, and Song Li. Viper-mcp: Detecting and exploiting taint-style vulnerabilities in model context protocol servers. *arXiv preprint arXiv:2605.21392*, 2026.

[15] Zhuoran Tan, Run Hao, Jeremy Singer, Yutian Tang, and Christos Anagnostopoulos. Mcp-sandboxscan: Wasm-based secure execution and runtime analysis for mcp tools. *arXiv preprint arXiv:2601.01241*, 2026.

[16] Mina Taraghi, Mohammad Mehdi Morovati, and Foutse Khomh. Real faults in model context protocol (mcp) software: a comprehensive taxonomy. *arXiv preprint arXiv:2603.05637*, 2026.

[17] Benny Toeppe, Amine Barrak, and Emna Ksontini. A large-scale dataset of MCP implementations on GitHub. *arXiv preprint arXiv:2607.10123*, 2026.

[18] Huijun Zhou, Xiaohan Zhang, Haozhe Zhang, Haoyang Zhang, Mi Zhang, and Min Yang. A first measurement study on authentication security in real-world remote mcp servers. *arXiv preprint arXiv:2605.22333*, 2026.